Campus Connect - Project Summary

Real-Time Client-Server Text Chat Application

Team: Marwan, Karim, Mohamed, Sara

Overview

Campus Connect is a real-time client-server chat application built for the CSCE 1102 Fundamentals of Computing II Lab capstone project. The system allows multiple users to connect to a central server, exchange public messages, start private chats, and communicate inside chat groups. The project focuses on a working GUI client, asynchronous networking, JSON-based communication, separated software layers, and automated unit testing.

Architecture and Technology

The client side is a Qt Widgets desktop application. The GUI layer handles screens, forms, buttons, status labels, and user interaction. Business logic is handled separately through controller/state classes, while networking is accessed through an INetworkClient interface. The real network client uses Qt Network and TCP sockets.
The server side is a command-line application implemented with Boost.Asio. It accepts multiple clients asynchronously, manages sessions, routes messages, handles disconnects, and communicates with clients using structured JSON messages over TCP.

Main Features

- Login screen with username, server IP, and port fields.
- Public chat between connected users.
- Online user list and private chat windows.
- Group creation, joining, and group messaging.
- Typing status updates and visible user feedback.
- JSON serialization/deserialization for all main message types.
- File/media attachment support as an extended feature.

Requirement Coverage

The project satisfies the required GUI component by providing multiple Qt screens, user input controls, dynamic updates, and feedback for user actions. It satisfies the networking component by using TCP sockets, asynchronous I/O, structured JSON messages, and separated networking code. It satisfies the testing component by using GoogleTest/GoogleMock with mocked client networking and server-side logic tests.

Bonus Features

The implemented bonus-level features include multiple users with private chats, group chat support, and attachment/media functionality. These features extend the application beyond a basic text-only chat and make it closer to a complete communication product.

Team Contributions

Marwan worked on networking, server-side functionality, message routing/session handling, and the bonus features. Karim worked on Qt Designer integration, interface polishing, and communication validation. Mohamed worked on Qt Designer integration, integration testing, final system testing, and debugging. Sara worked on Qt Designer integration, communication validation, presentation slides, and final delivery material.

Demo Plan

For the live demo, one machine runs the server executable, and multiple client applications connect using the server machine IP address and port 12345. The demo should show successful connection, public chat, private chat, group chat, and basic error handling. If network restrictions occur, the fallback is to run multiple clients locally or connect all laptops through a phone hotspot.

Conclusion

Campus Connect is a complete client-server chat system that demonstrates GUI design, networking, JSON communication, state management, testing, and integration. The project meets the core capstone requirements and includes additional bonus features that improve the final product.